

Phát triển ứng dụng web

ReactJS
(part 2)

Nội dung

- ☐ ReactJS – Lifecycle
- ☐ ReactJS – Hooks
- ☐ ReactJS – Side effects

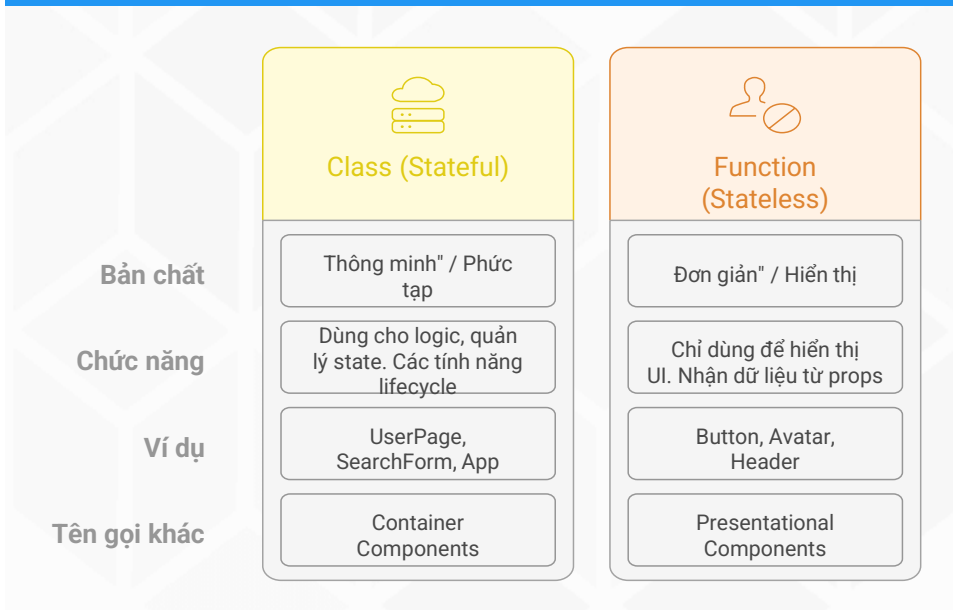
Nội dung

- ☐ **ReactJS – Lifecycle**
- ☐ ReactJS – Hooks
- ☐ ReactJS – Side effects

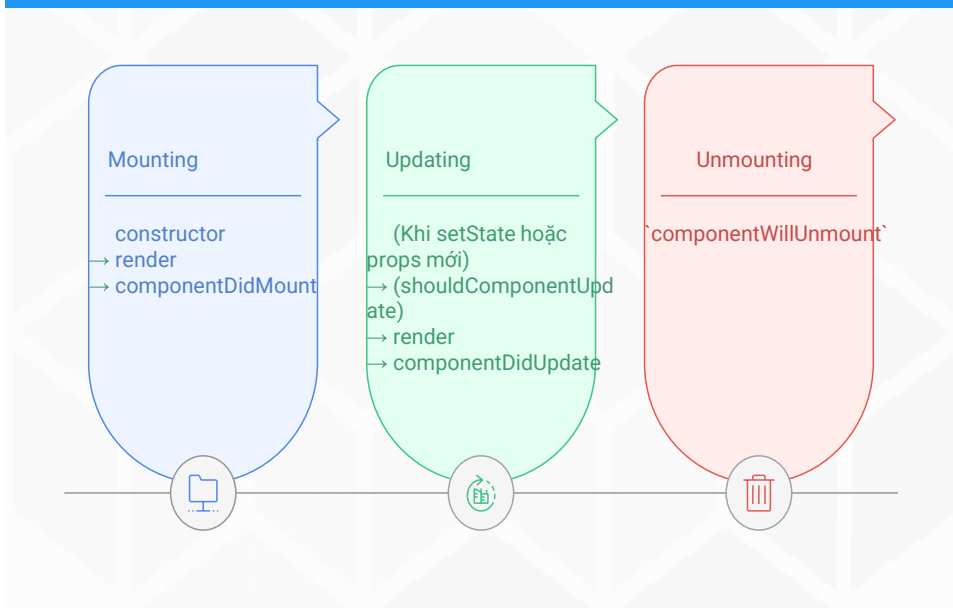
React cổ điển (trước React 16.8)

- ☐ State là một đối tượng JavaScript chứa dữ liệu **riêng** của một component. Khi **state** thay đổi, React sẽ tự động render lại (re-render) component đó để cập nhật UI.
- ☐ Class Component: Stateful, component có khả năng:
 - ☐ Khởi tạo và lưu trữ state (bộ nhớ).
 - ☐ Tự mình cập nhật state đó.
 - ☐ Kích hoạt re-render khi state thay đổi.
- ☐ Functional Component: Stateless
 - ☐ Chỉ là một hàm JavaScript đơn thuần.
 - ☐ Nhận `props` vào, trả về JSX (UI).
 - ☐ Không có **this**.
 - ☐ Không thể tự lưu trữ hay cập nhật dữ liệu.

Mô hình cổ điển (Trước React 16.8)



Class Component lifecycle



Từ React 16.8

❑ Functional Component không còn "stateless" nữa!

- ❑ Hook cho phép Functional Component có thể:
 - ❑ Khởi tạo và lưu trữ state.
 - ❑ Nhận một hàm để cập nhật state đó.
 - ❑ Kích hoạt re-render khi state thay đổi.

Thay đổi tư duy: Từ "Methods" đến "Phases"

Đặc điểm	Class Components (Cách cũ)	Functional Components (Cách mới)
Tư duy	"Làm gì khi sự kiện X xảy ra?"	"UI nên trông như thế nào với state hiện tại?"
Lifecycle	Các methods cụ thể (Methods)	Một luồng các giai đoạn (Phases)
Ví dụ	componentDidMount, componentDidUpdate	useState, useEffect (Hooks)
Trình tượng	Mệnh lệnh (Imperative)	Khai báo (Declarative)

ReactJS – Lifecycle trong Functional Component

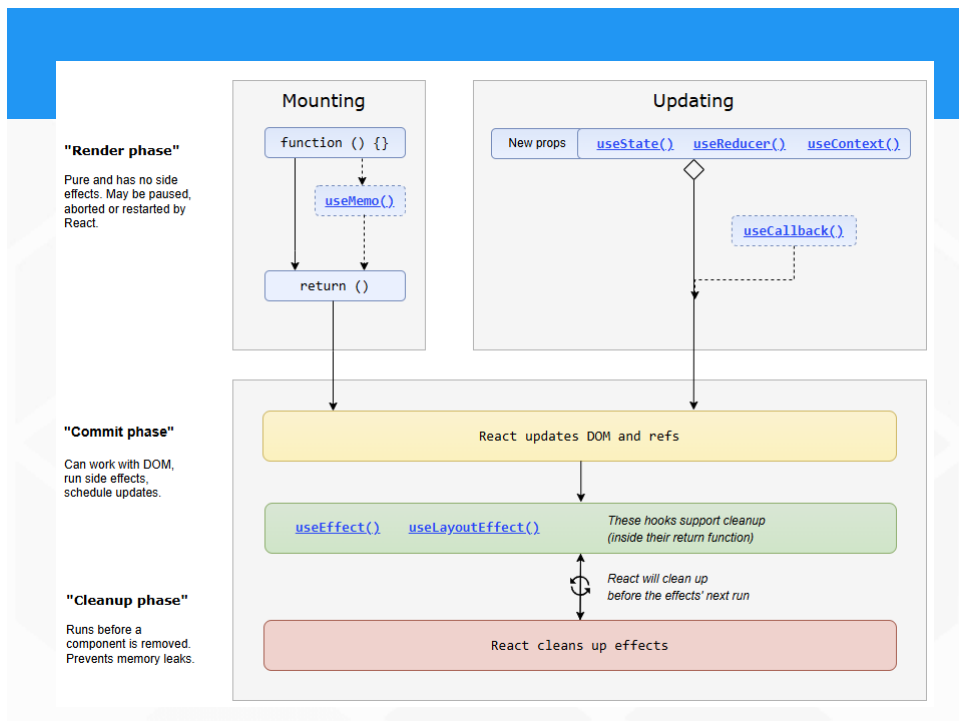
- ☐ Lifecycle là **toàn bộ quá trình React dùng để đồng bộ hóa state với DOM.**
- ☐ Hai pha chính
 - ☐ Render phase (pure, có thể bị ngắt / lặp lại): giai đoạn kết xuất
 - ☐ Commit phase (synchronous, một lần duy nhất/commit): giai đoạn ghi
- ☐ Pha hiệu ứng (effect)
 - ☐ Xử lý side effect ở đây

Render Phase

- ☐ **Mục đích:** "Lên kế hoạch" hay "Tính toán" xem UI *nên* trông như thế nào.
- ☐ Quy trình:
 - ☐ React gọi function component.
 - ☐ Kết quả trả về là một cây **Virtual DOM (VDOM)**.
 - ☐ React so sánh VDOM mới này với VDOM cũ (quá trình "diffing").
 - ☐ Nó "lên danh sách" các thay đổi cần thiết.
- ☐ Tính chất quan trọng:
 - ☐ **Phải "Pure" (Thuần khiết):** Không có side effects (như fetch API, sửa DOM) được phép ở đây.
 - ☐ **Có thể bị gián đoạn:** React có thể *tạm dừng, hủy bỏ*, hoặc *chạy lại* Render Phase (ví dụ: trong Concurrent Mode) để ưu tiên các tác vụ quan trọng hơn.

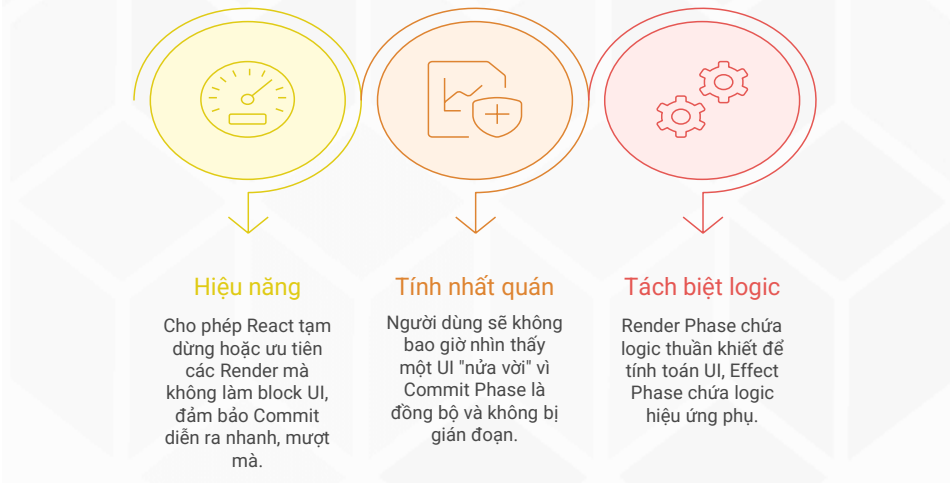
Commit Phase

- ❑ **Mục đích:** "Thi công" hay "Áp dụng" các thay đổi đã được lên kế hoạch (từ Render Phase) vào DOM thật.
- ❑ **Quy trình:**
 - ❑ React lấy "danh sách công việc" từ Render Phase.
 - ❑ Nó thực thi các thay đổi đó trên **DOM thật**.
- ❑ **Tính chất quan trọng:**
 - ❑ **Nhanh & Không bị gián đoạn:** Một khi đã bắt đầu, phase này sẽ chạy một mạch cho đến khi hoàn tất. Không thể tạm dừng.
 - ❑ **Nơi xảy ra Side Effects (cũ):** Đây là lúc các hook như `useLayoutEffect` chạy (ngay sau khi DOM update, nhưng trước khi trình duyệt vẽ).

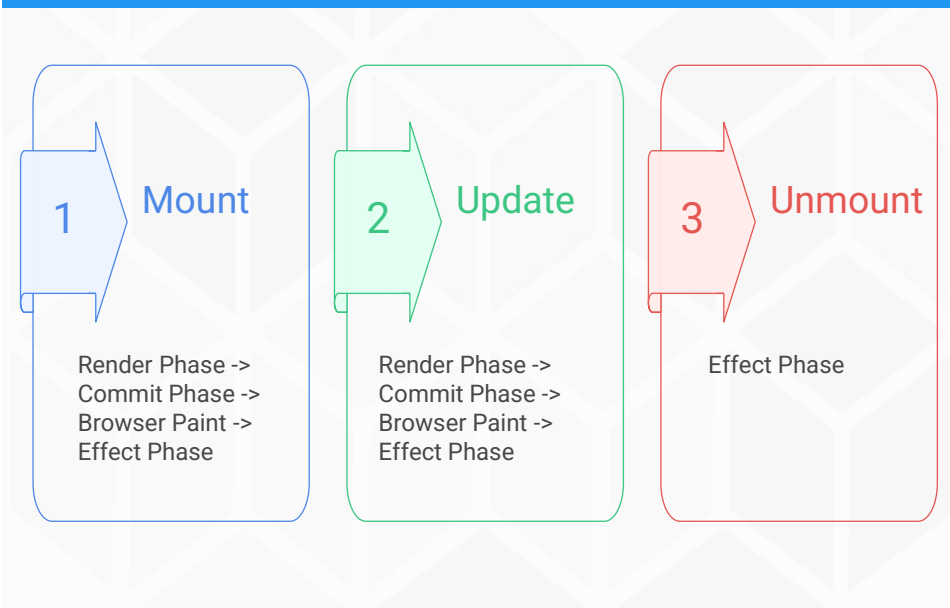


Tại sao chia làm 2 Giai đoạn?

- ❑ Việc tách biệt này là cốt lõi cho hiệu năng và khả năng của React:



Tương quan lifecycle truyền thống



React giúp UI tự cập nhật?

```
function Demo({ n }) {  
  return <div>Demo component with n = {n}</div>;  
}  
  
function DemoWrapper() {  
  const [n, setN] = useState(0);  
  let x = 10;  
  return (  
    <>  
      <Demo n={n} />  
      <button onClick={() => setN(n + 1)}>Increment n</button>  
    </>  
  );  
}
```

Khi nào React re-render?

- ☐ Khi **component** tự **setState**.
- ☐ Khi **cha re-render** và truyền **props khác** (reference hoặc giá trị khác).
- ☐ Khi **context** thay đổi.
- ☐ Hook/lifecycle không tự gây render trừ khi gọi **setState**.

Nội dung

- ❑ *ReactJS – Lifecycle*
- ❑ **ReactJS – Hooks**
- ❑ ReactJS – Side effects

Class Component

```
class Counter extends Component {
  constructor(props) {
    super(props);
    this.state = { count: 0 };
    // Phải bind 'this'
    this.handleIncrease = this.handleIncrease.bind(this);
  }
  componentDidMount() {
    document.title = `You clicked ${this.state.count} times`;
  }
  componentDidUpdate() {
    document.title = `You clicked ${this.state.count} times`;
  }
  handleIncrease() {
    this.setState({ count: this.state.count + 1 });
  }
  render() {
    return (
      <div>
        <p>You clicked {this.state.count} times</p>
        <button onClick={this.handleIncrease}>Click me</button>
      </div>
    );
  }
}
```

Hooks là gì?

- ❑ **Định nghĩa:** Hooks là các **hàm (functions)** cho phép "móc" (hook into) vào các tính năng của React như **state** và **lifecycle** từ bên trong **Function Components**.
 - ❑ Cho phép sử dụng các tính năng của React mà **không cần viết Class**.
- ❑ Không phá vỡ component model: vẫn là **pure render** → **commit**.
- ❑ **Không** phải là một "ma thuật": chỉ là hàm bắt đầu bằng **use*** có quy ước vận hành.
- ❑ Hooks là một sự bổ sung (additive). Chúng không thay thế Class Components ngay lập-tức, nhưng là cách được khuyến khích cho code mới.

Vì sao cần Hooks?

- ❑ Tránh **this** và phức tạp của class lifecycles.
- ❑ **Tách chia sẻ logic** không dính tới UI (custom hooks) thay vì mixin/HOC nặng nề.
- ❑ **Tổ chức code theo tính năng** (state + effect của cùng một tính năng nằm cạnh nhau) thay vì theo lifecycle rời rạc.
- ❑ **Tối ưu hoá:** transition, memoization...

Quy tắc Hooks (Rules of Hooks)

- ❑ **React dùng Linter (ESLint) để tự động bắt lỗi này.**
- ❑ Chỉ gọi Hooks ở top-level của component/custom hook (không trong **if**, **for**, hay **nested function**).
- ❑ **Chỉ gọi Hooks trong React function component hoặc custom hook.**
 - ❑ Lý do: React **rely on call order** để gán state/refs cho đúng hook.

Hooks theo nhu cầu

- ❑ State & Reducer: **useState**, **useReducer**.
- ❑ Side Effects & Lifecycle: **useEffect**, **useLayoutEffect**.
- ❑ Refs & Imperative: **useRef**, **useImperativeHandle**, **forwardRef**.
- ❑ Context: **useContext**.
- ❑ Memo hoá: **useMemo**, **useCallback**, **memo** (HOC).
- ❑ Concurrent & UX: **useTransition**, **useDeferredValue**.
- ❑ Other: **useId**, **useSyncExternalStore** (state ngoài React), **useInsertionEffect** (thư viện CSS-in-JS).

useState: trạng thái cục bộ

- ❑ State **được giữ** giữa các lần render.
- ❑ Setter **bất đồng bộ** theo batch; dùng **functional update** khi phụ thuộc state cũ.
- ❑ Set cùng giá trị → **không re-render**.

useReducer: state phức tạp

- ❑ Dùng khi state **hiều nhánh** hoặc cần **trace action**.
- ❑ Giúp **testable** và tách logic khỏi UI.

```
function reducer(state, action) {  
  switch (action.type) {  
    case "add":  
      return [...state, action.todo];  
    case "toggle":  
      return state.map((t) =>  
        t.id === action.id ? { ...t, done: !t.done } : t  
      );  
    default:  
      return state;  
  }  
}  
  
const [todos, dispatch] = useReducer(reducer, []);
```

useEffect: side effects sau paint

- ❑ Chạy **sau khi paint** (không chặn vẽ UI).
- ❑ Dùng cho: **I/O** (fetch, websocket), **subscriptions**, **DOM APIs** không cần layout sync, **logging**.
- ❑ **return** để **cleanup** khi deps đổi hoặc unmount.

```
useEffect(() => {  
  console.log("effect: chạy sau paint");  
  const id = setInterval(() => {}, 1000);  
  return () => clearInterval(id);  
});
```

useLayoutEffect: đồng bộ trước paint

- ❑ Chạy **sau commit nhưng trước paint** → có thể **chặn vẽ**.
- ❑ Dùng khi cần **đọc/ghi layout đồng bộ** (measure, scroll positioning).

```
function Counter({ count }) {  
  const boxRef = useRef(null);  
  useLayoutEffect(() => {  
    console.log(  
      "layoutEffect: đọc layout trước paint",  
      boxRef.current?.offsetWidth);  
    const start = performance.now();  
    while (performance.now() - start < 1000) {}  
  }, [count]);  
  return (  
    <>  
      <div  
        ref={boxRef}  
        style={{ width: 100 + count, height: 20, background: "#ddd" }}  
      />  
    </>  
  );  
}
```

Các Hooks phổ biến khác

- ❑ **useRef** lưu **mutable value** không gây re-render khi thay đổi.
- ❑ **useContext**: Sử dụng Context API của React (truyền data qua nhiều cấp component) mà không cần Context.Consumer.
- ❑ **useCallback** & **useMemo**: Dùng để tối ưu hóa hiệu năng (performance optimization).
- ❑ **useRef**: Truy cập trực tiếp vào một phần tử DOM, hoặc lưu trữ một giá trị "tham chiếu" không làm component re-render khi thay đổi.

Concurrent UX: useTransition & useDeferredValue

- ❑ **Transition**: đánh dấu update **không khẩn cấp** để UI vẫn mượt.
- ❑ **useDeferredValue(value)**: tạo bản sao trì hoãn để tránh lag khi value đổi liên tục.

Mapping lifecycle → Hooks

- ❑ Mount: `useEffect(..., [])` / `useLayoutEffect(..., [])`.
- ❑ Update: `useEffect/useLayoutEffect` với deps.
- ❑ Unmount: hàm cleanup trong effect.
- ❑ `getDerivedStateFromProps` → `useMemo/useEffect` đúng mục đích.

Custom Hooks: chia sẻ logic

- ❑ Quy ước: tên bắt đầu bằng **use**.
- ❑ Custom hook có thể gọi **các hook khác**.
- ❑ Đây **KHÔNG** phải là một tính năng của React. Đây là một *quy ước thiết kế* (design pattern) được kích hoạt bởi Hooks.

```
function useLocalStorage(key, initialValue) {  
  const [value, setValue] = useState(() => {  
    const v = window.localStorage.getItem(key);  
    return v ? JSON.parse(v) : initialValue;  
  });  
  
  useEffect(() => {  
    window.localStorage.setItem(key, JSON.stringify(value));  
  }, [key, value]);  
  return [value, setValue];  
}
```

Lợi ích của Hooks

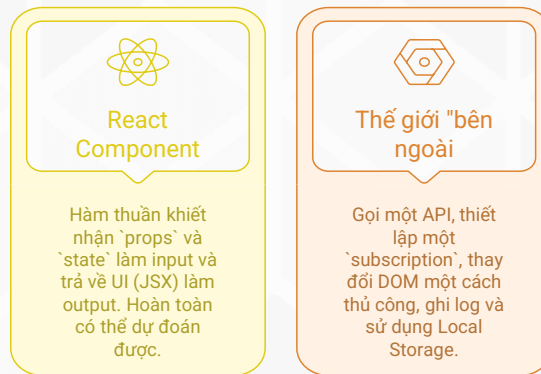
- ❑ **Code ngắn gọn, dễ đọc hơn:** Loại bỏ toàn bộ code rườm rà của Class (constructor, this, bind).
- ❑ Không còn **this**: Một trong những phần khó nhất của JavaScript đã biến mất khỏi component React.
- ❑ Tách biệt Logic.
- ❑ Tái sử dụng Logic dễ dàng (Custom Hooks)

Nội dung

- ❑ *ReactJS – Lifecycle*
- ❑ *ReactJS – Hooks*
- ❑ **ReactJS – Side effects**

"Side Effect" là gì?

- ❑ **Side Effect (Tác dụng phụ)** là bất kỳ logic nào trong component mà **tương tác với "thế giới bên ngoài"** — tức là, bất cứ điều gì không liên quan trực tiếp đến việc tính toán và trả về JSX.



Vấn đề: Đặt Side Effect ở đâu?

- ❑ Đặt ngay trong render phase

```
function UserProfile({ userId }) {  
  fetch(`https://api.example.com/users/${userId}`)  
    .then((res) => res.json())  
    .then((data) => {  
      // xử lý với data...  
    });  
  
  return (  
    <div>  
      <h1>Tên người dùng: ...</h1>  
    </div>  
  );  
}
```

- ❑ Component function sẽ chạy **mỗi khi component render lại** (do props hoặc state thay đổi).

Các tình huống khác

```
function BadListener() {  
  const onResize = () => console.log("resize", Date.now());  
  
  window.addEventListener("resize", onResize);  
  
  const [w, setW] = React.useState(0);  
  return <button onClick={() => setW(w + 1)}>Re-render ({w})</button>;  
}
```



```
function BadSetState() {  
  const [n, setN] = React.useState(0);  
  if (n < 5) setN(n + 1);  
  
  return <div>n = {n}</div>;  
}
```



```
function BadDOM() {  
  document.title = "Hello from render";  
  const width = window.innerWidth;  
  
  return <div>Width: {width}</div>;  
}
```



Giải pháp: Hook **useEffect**

- ❑ React cung cấp một "cánh cửa" đặc biệt để xử lý các side effect một cách an toàn: Hook **useEffect**.
- ❑ **useEffect** cho phép thực thi các side effect sau khi component đã render xong.
- ❑ Cách dùng

```
useEffect(  
  // 1. Callback Function:  
  //   Chứa logic side effect.  
  () => {  
    // Ví dụ: Thay đổi tiêu đề trang  
    document.title = `Bạn đã click ${count} lần`;  
  },  
  
  // 2. Dependency Array (Mảng phụ thuộc):  
  //   Quyết định KHI NÀO chạy lại effect.  
  [count]  
);
```

useEffect và Mảng Phụ thuộc (Dependency Array)

- ❑ Đây là phần **quan trọng nhất** để kiểm soát các side effect.

```
useEffect(() => {  
  const id = setInterval(() => {  
    setCount((c) => c + 1);  
  }, 1000);  
}, []);  
  
useEffect(() => {  
  const id = setInterval(() => {  
    setCount((c) => c + 1);  
  }, 1000);  
}, [count]);  
  
useEffect(() => {  
  const id = setInterval(() => {  
    setCount((c) => c + 1);  
  }, 1000);  
});
```

Dọn dẹp Side Effect (Effect Cleanup)

- ❑ Một số side effect cần được "dọn dẹp" khi component bị hủy (unmount).
 - ❑ Hủy một **subscription**.
 - ❑ Xóa một **timer** (clearInterval).
 - ❑ Hủy bỏ một **event listener** (removeEventListener).

```
useEffect(() => {  
  const timerId = setInterval(() => {}, 1000);  
  
  // Return một hàm dọn dẹp  
  return () => {  
    // Hàm này sẽ chạy khi component unmount  
    // hoặc trước khi effect chạy lại  
    clearInterval(timerId);  
  };  
}, []);
```

Nội dung

- ☐ *ReactJS – Lifecycle*
- ☐ *ReactJS – Hooks*
- ☐ *ReactJS – Side effect*
- ☒ **Bài tập**

Bài tập

- ☐ Với dữ liệu lấy được từ address: '<https://reqres.in>' sử dụng ReactJS để xây dựng ứng dụng web lấy dữ liệu có hỗ trợ tìm kiếm và phân trang.

